IOT BASED AUTOMATIC SWITCH USING ESP32

MINOR PROJECT REPORT

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY

in

ELECTRICAL ENGINEERING

SUBMITTED BY:

ATTAH U LLAH BEIGH

ROLL NO: 99230340573

UNDERGUIDANCE:

ER. RISHAV



BABA FARID COLLEGE OF ENGINEERING AND TECHNOLOGY,

BATHINDA – 151001

DEC - 2026

ISO 9001:2008 Certified

BABA FARID COLLEGE OF ENGG. & TECHNOLOGY

Muktsar Road, Bathinda-151001, Punjab (INDIA)

(Approved by AICTE, New Delhi and Affiliated to Maharaja Ranjit Singh Punjab Technical University, Bathinda)

NAAC GRADE A+ Accredited Institution (BFCET)

CERTIFICATE

I/we hereby certify that the work which is being presented in the project report entitled **"IOT BASED AUTOMATIC SWITCH"** by me/us in partial fulfillment of the requirements for the award of degree of **B.TECH (Electrical Engineering)** submitted in the Department of Electrical Engineering at **BABA FARID COLLEGE OF ENGINEERING AND TECHNOLOGY, BATHINDA** under **MAHARAJA RANJIT SINGH PUNJAB TECHNICAL UNIVERSITY, BATHINDA** is an authentic record of my/our own work carried out during the project period FROM "17 AUG2026 TO 11 NOV 2026"

Signature of the Students: -

ATTAH U LLAH BEIGH 99230340573

This is to certify that the above statement made by the candidates is correct to the best of my/our knowledge.

Signature of the Minor Project Guide/sDr.
MANISH KUMAR SINGLA

Head of Department (Dept. of Electrical Engineering)

Acknowledgment

We express our sincere gratitude to our project guide, **[Guide Name]**, for the valuable guidance, constant encouragement and support throughout the development of this project. His/her suggestions helped us to complete the work successfully.

We are thankful to the Head of the Department, **Manish Kumar Singla]**, and the Principal, Jyoti bansal **]**, for providing the laboratory facilities and a supportive academic environment. We also thank all the faculty and staff members of the department for their help.

Finally, we thank our parents and friends for their encouragement and cooperation during the completion of this project.

ATTAH U LLAH BEIGH; ROLL NO: 99230340573

Abstract

The Internet of Things (IoT) allows everyday electrical appliances to be monitored and controlled remotely through wireless networks. This project presents the design and implementation of an **IoT based automatic switch** using the **ESP32 DevKit** microcontroller and a **5V single-channel relay module**. The ESP32 creates its own Wi-Fi hotspot (Access Point mode) with the SSID "ATTAH_ESP32" and hosts a lightweight web server on the address 192.168.4.1. The user connects a smartphone or laptop to this hotspot, opens the address in any web browser, and presses the **ON** or **OFF** button of the web page "Smart Switch". The ESP32 then drives the relay through GPIO 23, which switches the connected load (an LED / lamp) on or off.

The system works without any external router or internet connection, uses very few components, is low cost, and is easy to extend. The project demonstrates a practical home-automation solution and forms a base for advanced smart-home applications.

Keywords: IoT, ESP32, Relay, Web Server, Wi-Fi Access Point, Home Automation.

Table of Contents

- Certificate
- Acknowledgment
- Abstract
- Table of Contents
- Chapter 1: Introduction
  - 1.1 Introduction
  - 1.2 Problem Statement
- Chapter 2: Literature Review
  - 2.1 Literature Review
  - 2.2 Objectives of the Project
- Chapter 3: Design Considerations
  a) 3.1 Details of the Components
  b) 3.2 Connection and Schematic Diagram
  c) 3.3 Principle of Operation / Working of Each Module
  d) 3.4 Actual Photo of the System
- Chapter 4: Results and Discussion
  - 4.1 Discussion on the Outcomes of the Project
- Chapter 5: Conclusions and Scope for Future Work
  a) 5.1 Conclusion
  b) 5.2 Future Scope
- Chapter 6: References
- Appendix

Chapter 1: Introduction

1.1 Introduction

In the modern world, automation is becoming a part of daily life. Traditional wall switches need a person to be physically present near the appliance. With the growth of the Internet of Things (IoT), it is now possible to connect electrical devices to a network and control them from a smartphone.

IoT is a network of physical objects embedded with sensors, electronics, software and connectivity, which enables them to collect and exchange data and to be controlled remotely. Home automation is one of the most popular applications of IoT, where lights, fans and other appliances are switched through a mobile phone or computer.

This project uses the **ESP32**, a low-cost, low-power microcontroller with built-in Wi-Fi and Bluetooth, developed by Espressif Systems. The ESP32 is connected to a **relay module**, which acts as an electrically operated switch for the load. A web page hosted on the ESP32 itself provides ON and OFF buttons. Because the ESP32 works as its own Wi-Fi Access Point, the user does not need a home router or internet service to control the load.

1.2 Problem Statement

Conventional switches have the following limitations:

1. The user must be physically near the switch to operate it.
2. They are inconvenient for elderly or physically challenged people.
3. Appliances are often left ON by mistake, wasting electrical energy.
4. Many commercial smart switches are expensive and depend on a router, internet connection or cloud service.

Hence, there is a need for a **simple, low-cost and reliable wireless switch** that can be operated from a smartphone browser without any special application and without depending on an internet connection. This project aims to design such a system using the ESP32 and a relay module.

Chapter 2: Literature Review

2.1 Literature Review

Home automation has evolved over several generations of technology:

1. **Wired and X10 systems:** Early automation used power-line communication (such as X10) to control appliances. These systems were slow, prone to noise and had limited features.
2. **GSM / DTMF based control:** Appliances were controlled by SMS or phone calls using a GSM modem and microcontroller. This works over long distances, but has recurring SMS cost and slow response.
3. **Bluetooth based control:** Modules such as HC-05 with Arduino allow control from a phone application. The range is limited to about 10 metres and a dedicated app is normally required.
4. **Wi-Fi based control with ESP8266:** The ESP8266 made Wi-Fi control cheap, but it has limited GPIO pins, no Bluetooth and lower processing power.
5. **ESP32 based IoT systems:** The ESP32 is a dual-core 32-bit microcontroller with integrated Wi-Fi, Bluetooth, many GPIO pins, ADC, touch sensors and low-power modes. It can host a web server, connect to cloud platforms (such as Blynk, MQTT or Firebase) and run in Station, Access Point or combined modes.

Various researchers and hobbyists have implemented relay control using ESP32 through cloud dashboards, mobile applications and voice assistants. Most of these depend on a router and internet connection. The present project follows a **local web-server approach in Access Point mode**, which removes the dependency on the router and internet, and keeps the design simple and secure with a WPA2 password.

Technology	Range	Needs Internet	Cost	Remarks
GSM / SMS	Very long	No (needs SIM)	Medium	Recurring SMS charges
Bluetooth (HC-05)	~10 m	No	Low	App required
ESP8266 Wi-Fi	~30-50 m	Optional	Low	Limited pins
ESP32 Wi-Fi (this project)	~30-50 m	No (AP mode)	Low	Browser only, no app

2.2 Objectives of the Project

[1] To design and build a wireless automatic switch using the ESP32 microcontroller.
[2] To control an electrical load (LED / lamp) using a relay module.
[3] To create a Wi-Fi hotspot from the ESP32 so that no router or internet is required.
[4] To develop a mobile-friendly web page with ON and OFF buttons for controlling the load.
[5] To display the live status (ON / OFF) of the load on the web page.
[6] To keep the system low cost, compact and easy to expand.

Chapter 3: Design Considerations

3.1 Details of the Components

The system consists of the following blocks: **Power Supply, ESP32, Connecting Wires, Relay Module, and LED (Load)**.

1. Power Supply

The ESP32 DevKit board is powered through its USB Type-C port from a 5V mobile charger, power bank or computer USB port. The on-board AMS1117 3.3V regulator converts the 5V input to 3.3V for the ESP32 chip. The 5V line (VIN pin) is also used to power the relay module.

2. ESP32 DevKit (ESP32-WROOM-32)

Parameter	Specification
Microcontroller	ESP32-WROOM-32 module (Xtensa dual-core 32-bit LX6)
Clock speed	Up to 240 MHz
Wireless	Wi-Fi 802.11 b/g/n (2.4 GHz) and Bluetooth
Operating voltage	3.3V (logic)
Input voltage	5V via USB Type-C / VIN
USB-to-serial chip	Silicon Labs CP2102
Voltage regulator	AMS1117 (3.3V)
Buttons	EN (reset) and BOOT
GPIO	Up to 30+ pins (digital I/O, ADC, DAC, PWM, I2C, SPI, UART)

The ESP32 is the brain of the system. It runs the program, creates the Wi-Fi hotspot, hosts the web server and controls the relay pin.

3. Relay Module (5V, 1-Channel)

The relay module uses a **Songle SRD-05VDC-SL-C** relay.

Parameter	Specification
Coil voltage	5V DC

Parameter	Specification
Contact rating	10A at 250V AC / 10A at 30V DC
Contacts	COM, NO (normally open), NC (normally closed)
Input pins	VCC, GND, IN
Indicators	Green LED (relay status) and Red LED (power)
Trigger type	Active LOW
Protection	Flyback diode across the coil, transistor driver

The relay isolates the low-voltage control circuit from the load circuit. A small signal from the ESP32 can therefore switch a much larger load.

4. Connecting Wires

Male-to-female jumper wires connect the ESP32 pins to the relay module header. Insulated wires are used for the load circuit.

5. LED / Lamp (Load)

An LED (with a suitable series resistor on a low-voltage supply) is used as the load to demonstrate switching. The same arrangement can control an AC lamp when connected through the relay contacts, with proper insulation and safety precautions.

6. Software Tools

1. Arduino IDE with ESP32 board package
2. Libraries: WiFi.h and WebServer.h
3. Web browser on a smartphone or laptop

3.2 Connection and Schematic Diagram

Block diagram:

Power Supply (5V USB) → ESP32 DevKit → Connecting Wires → Relay Module → LED / Lamp (Load)

Pin connections:

From	To	Purpose
ESP32 VIN (5V)	Relay VCC	Power for relay coil
ESP32 GND	Relay GND	Common ground
ESP32 GPIO 23 (D23)	Relay IN	Control signal
Relay COM	Supply positive (+)	Common terminal of load supply

From	To	Purpose
Relay NO	LED / Lamp (+)	Load turns on when relay is energised
LED / Lamp (-)	Supply negative (-)	Return path

[Insert circuit / schematic diagram here - can be drawn in Fritzing, Proteus or EasyEDA]

Safety note: When controlling mains (230V AC) loads, the wiring must be insulated and enclosed, and it should be done under the supervision of the guide. For demonstration, a low-voltage LED circuit is recommended.

3.3 Principle of Operation / Working of Each Module

Power supply module: Provides regulated 5V to the ESP32 board. The onboard regulator produces 3.3V for the microcontroller.

ESP32 module: On power-up the program runs `setup()`. The relay is first kept OFF, then the Wi-Fi is set to Access Point mode with `WiFi.softAP()`. A hotspot named **ATTAH_ESP32** with password **987654321** (minimum 8 characters, WPA2) is created. The default IP address of the ESP32 is **192.168.4.1**. The web server then starts on port 80, and in `loop()` the function `server.handleClient()` continuously listens for browser requests.

Web server module: Three routes are defined:

URL	Action
`/`	Displays the "Smart Switch" page with the present status and ON / OFF buttons
`/on`	Calls `relayOn()` and redirects back to the home page
`/off`	Calls `relayOff()` and redirects back to the home page

Relay module: The relay input is **active LOW**. When GPIO 23 is set as OUTPUT and driven LOW, the transistor energises the relay coil, the green LED glows, and the COM-NO contacts close, so the load turns ON. In the OFF condition, the pin is changed to INPUT (high impedance), which lets the relay module's input pull-up keep the relay de-energised, so the contacts open and the load turns OFF. This method also ensures the relay stays OFF during boot-up.

Load (LED / lamp): Receives power only when the relay contacts are closed.

Working steps:

1. Power the ESP32 using the USB Type-C cable.
2. ESP32 starts the Wi-Fi hotspot "ATTAH_ESP32".
3. Connect the phone to this hotspot using the password.
4. Open a browser and type **192.168.4.1**.

5. Press **ON** - the relay switches ON and the page shows "Lamp: ON".
6. Press **OFF** - the relay switches OFF and the page shows "Lamp: OFF".

Flowchart: Start → Initialise relay OFF → Start Wi-Fi Access Point → Start web server → Wait for client request → (ON request → Relay ON) / (OFF request → Relay OFF) → Update web page → back to Wait.

3.4 Actual Photo of the System

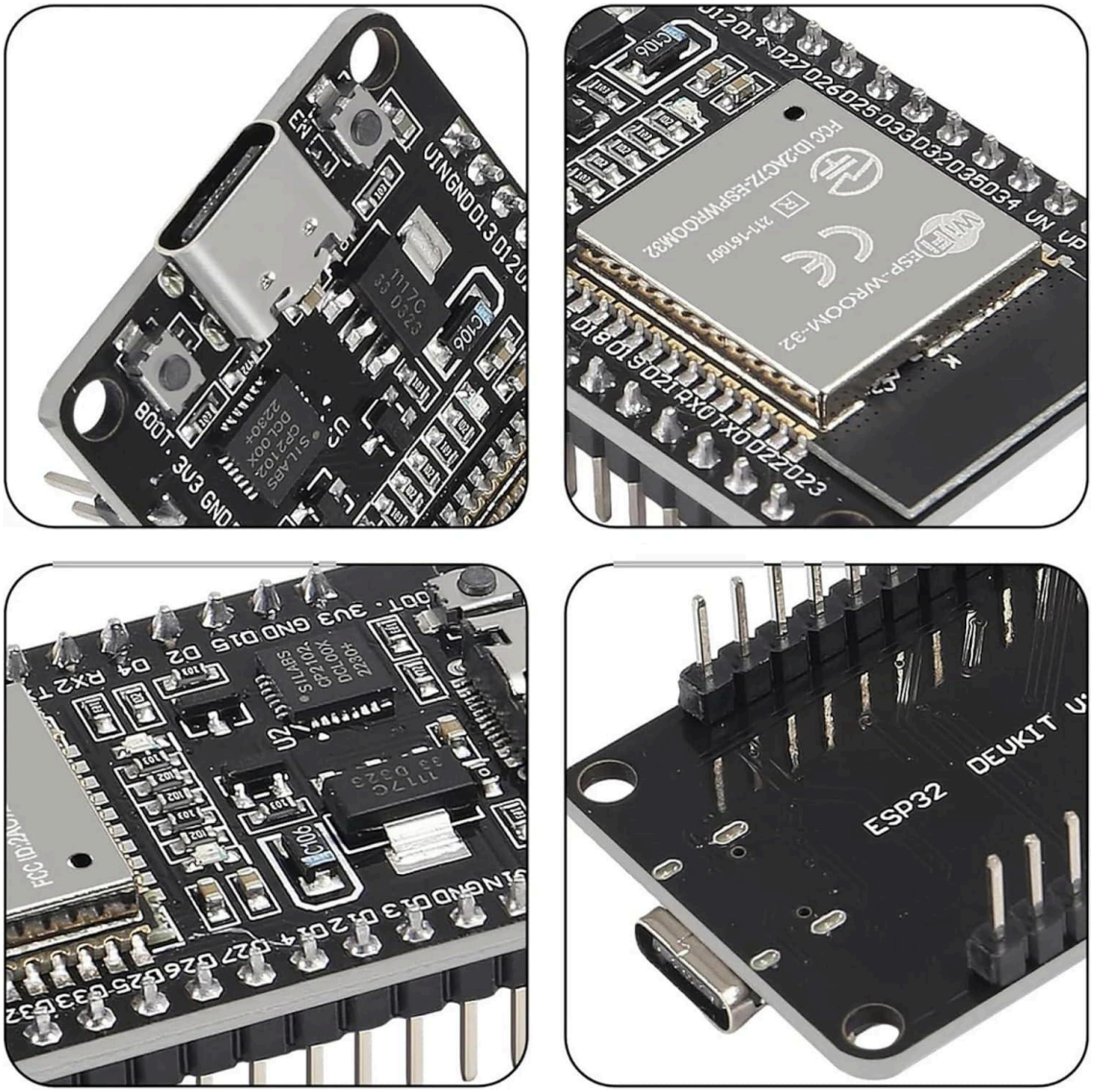
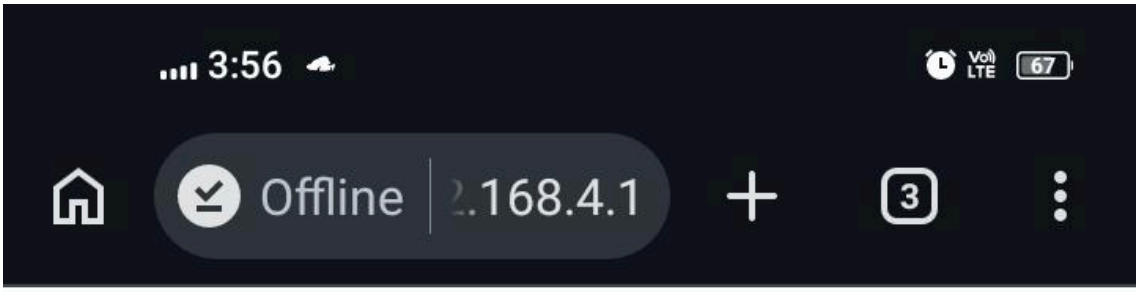


Figure 3.1: Complete hardware setup (Power supply, ESP32, wires, relay and LED)

[Insert screenshot of the "Smart Switch" web page here]



Smart Switch

Lamp: OFF

ON

OFF

Figure 3.2: Web interface showing ON and OFF buttons

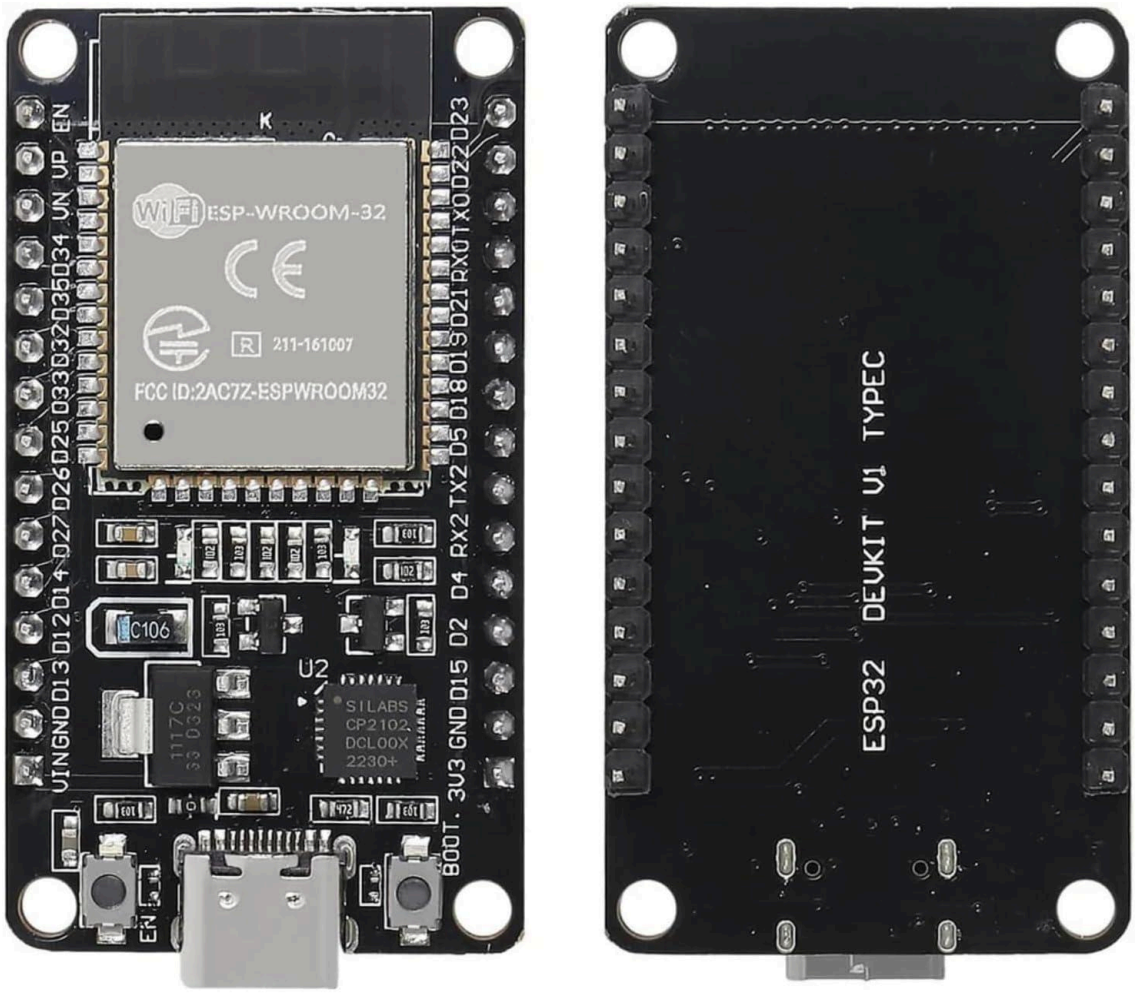


Figure 3.3: ESP32 DevKit board

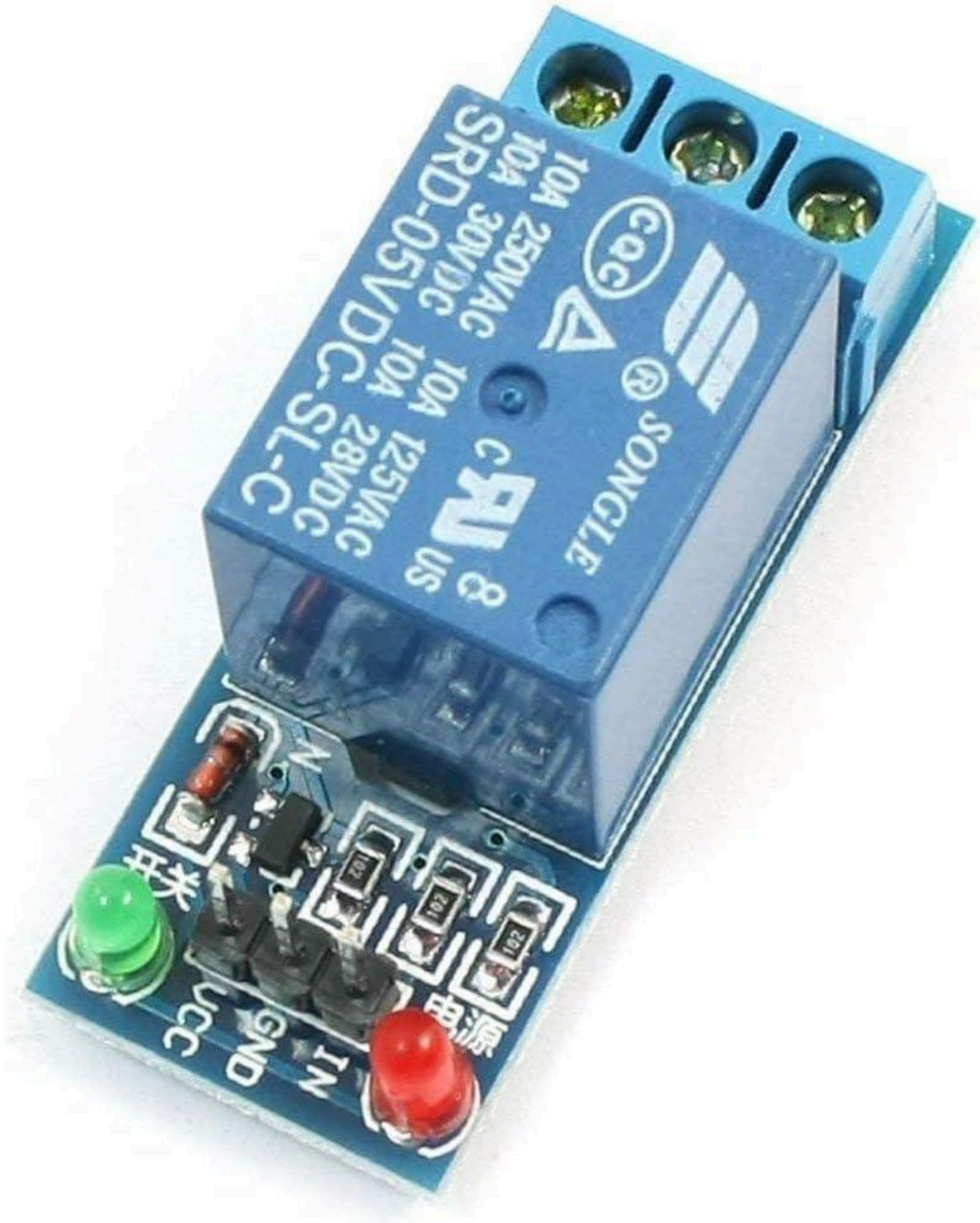


Figure 3.4: 5V single-channel relay module

Chapter 4: Results and Discussion

4.1 Discussion on the Outcomes of the Project

The system was assembled and tested successfully. The following observations were made:

- After power-up, the ESP32 created the hotspot "ATTAH_ESP32" within a few seconds, and the serial monitor displayed "Hotspot started" and the IP address 192.168.4.1.
- A smartphone connected to the hotspot and opened the web page without any application.
- Pressing **ON** switched the relay ON (the relay clicked and the green LED glowed), and the connected LED / lamp turned ON. The page status changed to "Lamp: ON".
- Pressing **OFF** switched the relay OFF and the load turned OFF. The page status changed to "Lamp: OFF".
- The response time was almost instantaneous (less than one second) within the Wi-Fi range.
- The relay remained OFF at start-up, avoiding accidental switching of the load.

Test	Expected Result	Observed Result
Hotspot creation	SSID visible on phone	Pass
Open 192.168.4.1	Web page loads	Pass
Press ON	Relay and load ON	Pass
Press OFF	Relay and load OFF	Pass
Status display	Correct ON / OFF text	Pass
Power-up state	Load OFF	Pass

Advantages:

- Works without router or internet.
- No mobile application is needed; any browser can be used.
- Low cost and compact design.
- Password-protected hotspot.
- Easy to modify and expand.

Limitations:

- Control is possible only within the Wi-Fi range (approximately 30-50 metres).
- The phone must disconnect from its regular Wi-Fi/mobile data connection to the hotspot while controlling.
- Only one relay (one load) is controlled at present.
- The relay state is not stored after a power cut.

Chapter 5: Conclusions and Scope for Future Work

5.1 Conclusion

An IoT based automatic switch was successfully designed and implemented using the ESP32 and a relay module. The ESP32 works as a Wi-Fi Access Point and hosts a web page through which a user can turn a load ON or OFF from a smartphone or laptop browser. The project met all its objectives: it is simple, low cost, works without internet, and gives reliable switching of the connected load. It demonstrates how IoT can be used for practical home automation and energy saving.

5.2 Future Scope

- **Internet / cloud control:** Connect the ESP32 to a home router (Station mode) and use MQTT, Blynk or Firebase for control from anywhere in the world.
- **Multi-channel control:** Use a 4 or 8 channel relay module to control several appliances.
- **Voice control:** Integration with Google Assistant or Amazon Alexa.
- **Scheduling and timers:** Automatic ON/OFF at preset times using an RTC or NTP time.
- **Sensor based automation:** Add LDR, PIR, temperature or humidity sensors for automatic control.
- **State memory:** Store the last relay state in flash memory (EEPROM / Preferences).
- **Energy monitoring:** Add a current sensor to measure power consumption.
- **Dedicated mobile app and OTA updates:** Develop an Android app and update firmware wirelessly.
- **Enhanced security:** Add login authentication and HTTPS to the web interface.

Chapter 6: References

1. Espressif Systems, "ESP32 Series Datasheet" and "ESP32-WROOM-32 Datasheet". https://www.espressif.com
2. Espressif Systems, "Arduino core for the ESP32 - WiFi and WebServer library documentation". https://github.com/espressif/arduino-esp32
3. Songle Relay, "SRD-05VDC-SL-C Relay Datasheet".
4. Arduino, "Arduino IDE and Language Reference". https://www.arduino.cc
5. Random Nerd Tutorials, "ESP32 Web Server and Relay Control Tutorials". https://randomnerdtutorials.com
6. Silicon Labs, "CP2102 USB to UART Bridge Datasheet".
7. [Add any books / research papers / websites used by you]

Appendix

Appendix A: Complete Source Code (Arduino IDE)

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "ATTAH_ESP32";
```

```
const char* password = "987654321";   // kam se kam 8 characters
const int RELAY_PIN = 23;

WebServer server(80);
bool relayState = false;

void relayOn() {
 pinMode(RELAY_PIN, OUTPUT);
 digitalWrite(RELAY_PIN, LOW);   // ON
 relayState = true;
}

void relayOff() {
 pinMode(RELAY_PIN, INPUT);      // OFF (jo trick aapke relay pe
chali)
 relayState = false;
}

void handleRoot() {
 String html = "<!DOCTYPE html><html><head>";
 html += "<meta name='viewport' content='width=device-width,
initial-scale=1'>";
 html += "<style>";
 html += "body{font-family:Arial;text-align:center;margin-top:50px;}";
 html += "button{font-size:32px;padding:20px
40px;margin:10px;border:none;border-radius:12px;color:white;}";
 html += ".on{background:#2e9e4f;} .off{background:#d33;}";
 html += "</style></head><body>";
 html += "<h1>Smart Switch</h1>";
 html += "<h2>Lamp: ";
 html += relayState ? "ON" : "OFF";
 html += "</h2>";
 html += "<a href='/on'><button class='on'>ON</button></a>";
 html += "<a href='/off'><button class='off'>OFF</button></a>";
 html += "</body></html>";
 server.send(200, "text/html", html);
}

void setup() {
 Serial.begin(115200);
```

```
  relayOff();

  WiFi.mode(WIFI_AP);
  WiFi.softAP(ssid, password);

  Serial.println("Hotspot started");
  Serial.print("IP: ");
  Serial.println(WiFi.softAPIP());    // 192.168.4.1

  server.on("/", handleRoot);
  server.on("/on", []() {
    relayOn();
    server.sendHeader("Location", "/");
    server.send(303);
  });
  server.on("/off", []() {
    relayOff();
    server.sendHeader("Location", "/");
    server.send(303);
  });
  server.begin();
}

void loop() {
  server.handleClient();
}
```

Appendix B: Code Explanation

Line / Function	Description
`#include <WiFi.h>`	Library for Wi-Fi functions of the ESP32
`#include <WebServer.h>`	Library to create an HTTP web server
`ssid`, `password`	Name and password of the hotspot created by the ESP32
`RELAY_PIN = 23`	GPIO pin connected to the relay IN pin
`relayOn()`	Sets the pin as output and LOW, energising the relay (active LOW)

Line / Function	Description
`relayOff()`	Sets the pin as input (high impedance), de-energising the relay
`handleRoot()`	Builds and sends the HTML page with the current status and buttons
`WiFi.softAP()`	Starts the Access Point
`server.on()`	Registers the URL routes `/`, `/on`, `/off`
`server.handleClient()`	Handles incoming client requests continuously

Appendix C: Components List

S.No.	Component	Quantity
1	ESP32 DevKit (ESP32-WROOM-32, USB Type-C)	1
2	5V Single-channel relay module (SRD-05VDC-SL-C)	1
3	LED / Lamp with holder	1
4	5V USB power supply / adapter and Type-C cable	1
5	Jumper wires	As required
6	Resistor (for LED, if used on low voltage)	1